

Instalación de Docker

[1.Instalación de Docker en Ubuntu](#)

[2. Gestión de Contenedores \(docker run / docker exec\)](#)

[3. Administración y control de Contenedores](#)

[4. Gestión de Imágenes \(docker image\)](#)

[4. Mantenimiento y Limpieza del Sistema](#)

[5. Gestión con Docker Compose \(docker compose\)](#)

1.Instalación de Docker en Ubuntu

❑# **Añade la clave GPG oficial de Docker:**

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

❑

❑# **Añadir el repositorio a las fuentes de APT:**

```
echo \
  "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
```

❑

❑# **Instalar los paquetes necesarios:**

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
```

❑

❑# **Verificar que el servicio esté activo:**

```
sudo systemctl status docker
```

❑Deberías ver el estado resaltado en verde como **active (running)**.

❑# **Ejecutar Docker sin 'sudo' (opcional):**

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

□ Ejecuta `docker run hello-world` sin anteceder `sudo`. Deberías ver un mensaje de bienvenida confirmando la instalación.

2. Gestión de Contenedores (`docker run` / `docker exec`)

El comando `docker run` crea e inicia un contenedor. Sus opciones más comunes son:

- `-d` (detached): Ejecuta el contenedor en segundo plano.
- `-p [puerto_host]:[puerto_contenedor]` (ports): Mapea puertos entre tu máquina y el contenedor (ejemplo: `-p 8080:80`).
- `-v [ruta_host]:[ruta_contenedor]` (volumes): Monta un volumen o directorio del sistema host en el contenedor para persistir datos (ejemplo: `-v /mi/data:/var/lib/mysql`).
- `--name [nombre]` (name): Asigna un nombre personalizado al contenedor.
- `-e [VARIABLE=valor]` (environment): Define variables de entorno dentro del contenedor.
- `--rm` (remove): Elimina automáticamente el contenedor cuando este se detiene.
- `-it` (interactive + tty): Abre una terminal interactiva (muy usado con `docker exec`).

Ejemplos prácticos:

Levantar un contenedor MySQL en segundo plano exponiendo puertos y variables

```
docker run -d --name mi-mysql -p 3306:3306 -e MYSQL_ROOT_PASSWORD=secreto mysql:latest
```

Entrar a la terminal de un contenedor en ejecución

```
docker exec -it mi-mysql bash
```

3. Administración y control de Contenedores

Comandos esenciales para inspeccionar y controlar el estado de los contenedores:

- `docker ps`: Muestra los contenedores en ejecución.
 - `docker ps -a`: Muestra todos los contenedores (activos y detenidos).
- `docker logs [contenedor]`: Muestra los registros de salida del contenedor.
 - `docker logs -f [contenedor]`: Sigue los registros en tiempo real (*tail/follow*).
- `docker stop [contenedor]`: Detiene un contenedor de forma segura.
 - `docker kill [contenedor]`: Fuerza el apagado inmediato del contenedor.
- `docker start [contenedor]`: Inicia un contenedor previamente detenido.
- `docker rm [contenedor]`: Elimina un contenedor detenido.
 - `docker rm -f [contenedor]`: Fuerza la eliminación de un contenedor, incluso si está en ejecución.

4. Gestión de Imágenes (**docker image**)

Opciones para descargar, listar y limpiar imágenes del sistema:

- **docker pull [imagen]**: Descarga una imagen desde Docker Hub.
- **docker images** (o **docker image ls**): Lista todas las imágenes almacenadas localmente.
- **docker rmi [imagen]** (o **docker image rm**): Elimina una imagen local.
- **docker build -t [nombre:tag] .**: Construye una imagen a partir de un archivo **Dockerfile** presente en el directorio actual.

4. Mantenimiento y Limpieza del Sistema

Para liberar espacio en disco eliminando recursos obsoletos o en desuso:

- **docker system prune**: Elimina contenedores detenidos, redes no utilizadas e imágenes sin etiqueta (*dangling*).
- **docker system prune -a --volumes**: Limpieza profunda. Elimina todos los contenedores detenidos, imágenes no usadas y volúmenes no asociados.
- **docker stats**: Muestra en tiempo real el consumo de CPU, memoria y red de todos los contenedores activos.

5. Gestión con Docker Compose (**docker compose**)

Cuando se definen múltiples servicios en un archivo **docker-compose.yml**:

- **docker compose up -d**: Levanta todos los servicios definidos en segundo plano.
- **docker compose down**: Detiene y elimina los contenedores, redes y volúmenes creados por el compose.
- **docker compose logs -f [servicio]**: Muestra los logs de un servicio específico.
- **docker compose ps**: Lista los contenedores asociados a la pila actual.